

Can a small-array switch make merge sort faster?

Hybrid merge sort + insertion sort

Choose a threshold S . Measure both CPU time and key comparisons.

n = input length S = insertion-sort threshold

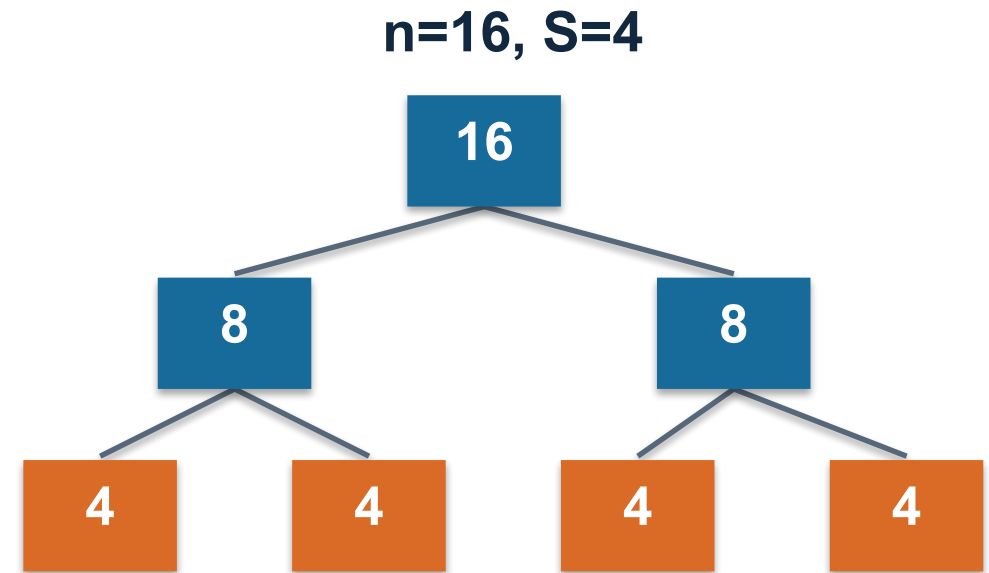
Research question: does the fastest S also minimize comparisons?

1.147×

measured speedup
on fresh 10m-element
arrays

Sort small leaves; merge the sorted halves

```
hybrid(A, left, right, S):  
    if right-left <= 1: return  
    if right-left <= S:  
        insertionSort(A, left, right)  
        return  
    mid = left + (right-left)/2  
    hybrid(A, left, mid, S)  
    hybrid(A, mid, right, S)  
    merge(A, left, mid, right)
```



Insertion sort at the four leaves

Sorted leaves → sorted halves → sorted whole

Stable ties • one reusable buffer

The threshold trades leaf work for merge levels

Insertion-sort leaves

About n/S leaves $\times O(S^2)$ work per leaf

Merge levels

About $\log_2(n/S)$ levels $\times O(n)$ work per level

$$T(n, S) = O(nS + n \log_2(n/S))$$

Worst-case bound for $1 \leq S \leq n$; not an exact random-input count.

Fixed S

$\Theta(n \log n)$ worst case

$S = 1$

Ordinary merge sort

$S \geq n$

Insertion sort: $O(n^2)$ worst case

Experiment flow: from generated arrays to a fair comparison

Part (b) creates the inputs. We then use those inputs to choose S and test it on fresh data.

01 GENERATE

Uniform integers in $[1, 10^7]$
9 sizes: 1,000 → 10,000,000

Why: isolate the effect of n

02 TUNE S

Sweep candidate thresholds
5 seeds × 3 timing repeats
Choose the lowest median CPU time
Why: find a fast S before testing it

03 VALIDATE

Lock $S = 64$ before testing
5 fresh 10m-element arrays
Compare merge sort vs hybrid
Why: avoid choosing S on test data

CONTROLS HELD CONSTANT

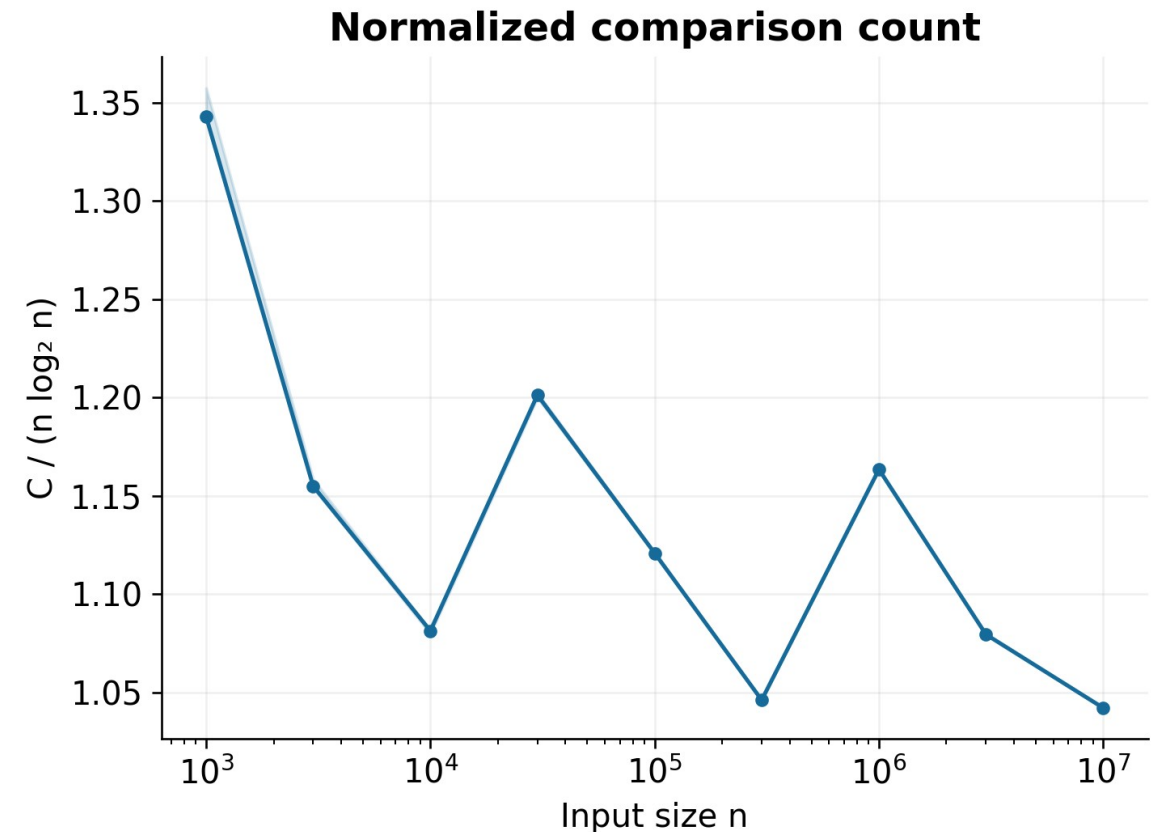
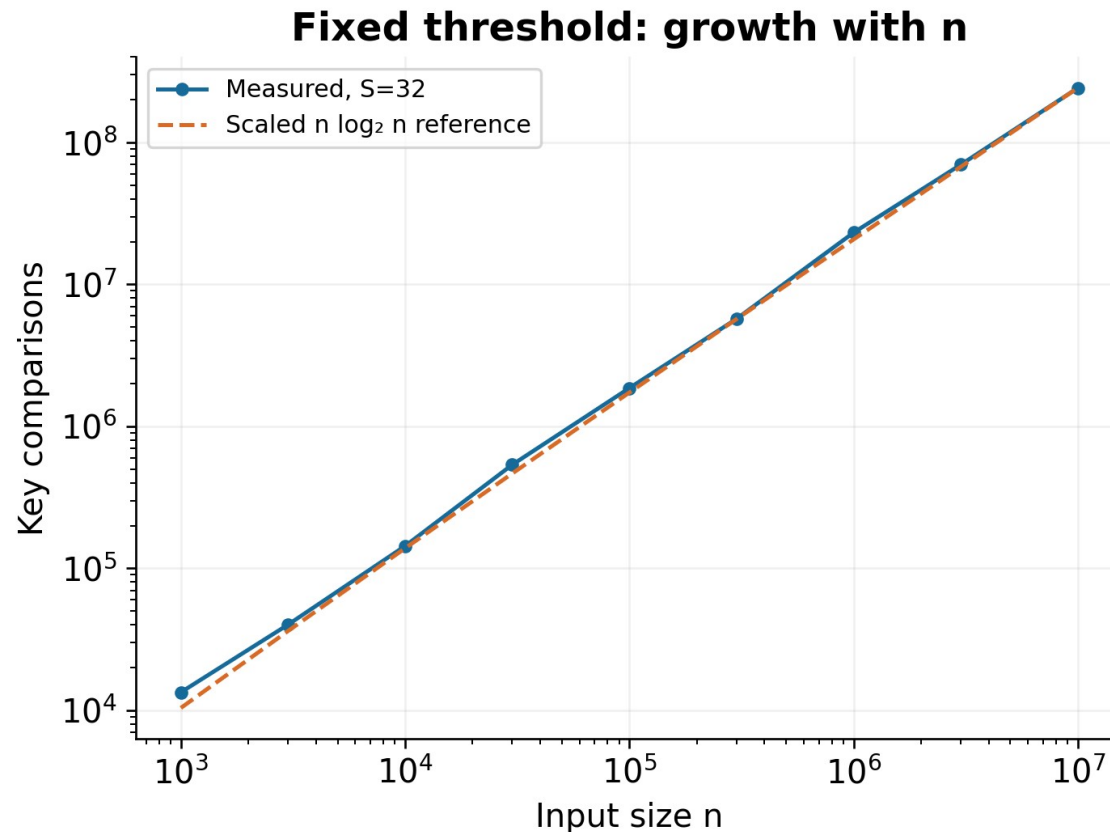
Same array for every candidate

Warm-up + shuffled run order

Sort only is timed; counters run separately

Every output checked against `std::sort` • Apple M5 / C++17 / -O3

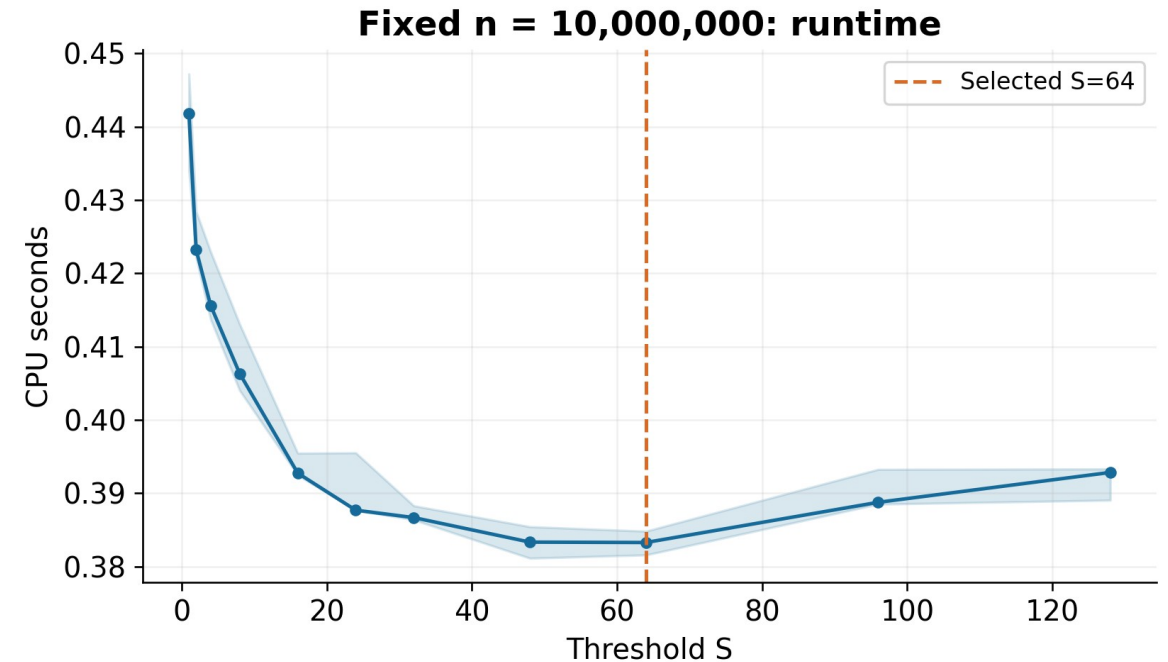
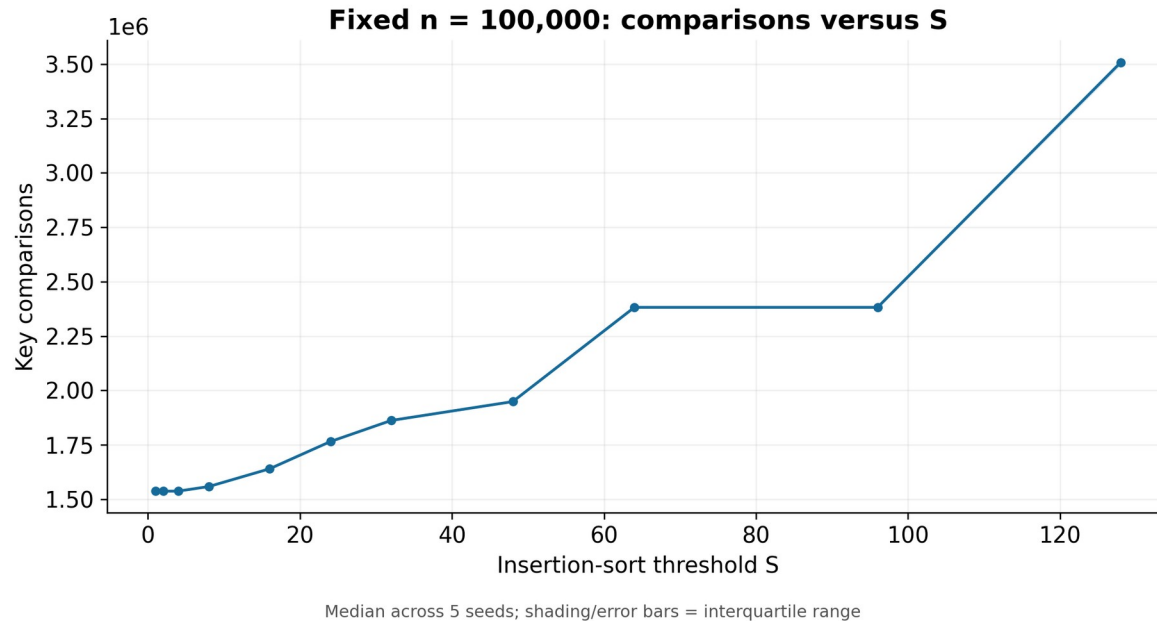
Fixed S: comparison growth agrees with $n \log n$



Median across 5 seeds; shading/error bars = interquartile range

$S=32$ • normalized ratio 1.042–1.343 • finite-size leaf effects explain the variation

Fastest runtime does not mean fewest comparisons



Selected S=64

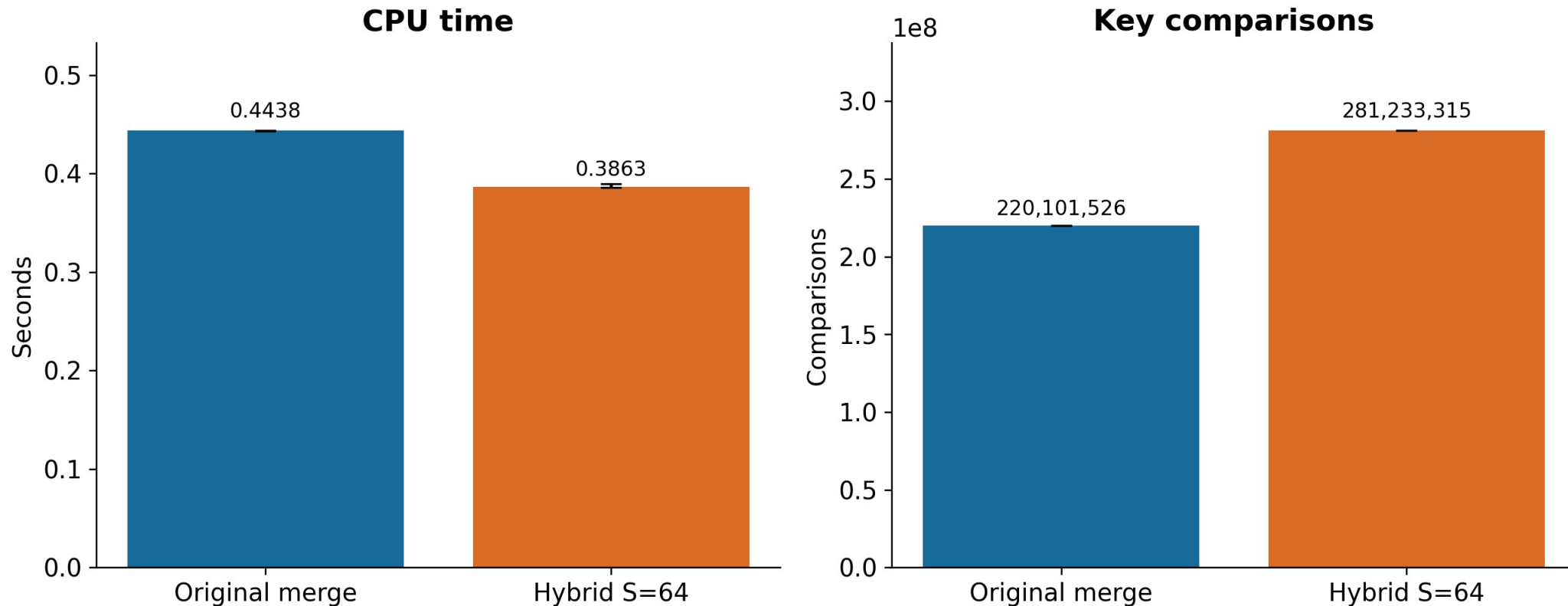
Minimum comparisons at 10m:
S=1

Coarse winners by n

10k: 48 · 100k: 96 · 1m: 32 · 10m: 64

Hybrid S=64: 1.147× speedup with 27.8% more comparisons

Held-out n=10,000,000: paired median speedup 1.147×; comparisons +27.8%



Median across 5 seeds; shading/error bars = interquartile range

Speedup = median of per-seed time ratios • IQR: 1.138–1.152× • 5 fresh datasets

Correctness, scaling, and a measured runtime benefit

LIVE DEMO

```
./build/demo
```

```
Input:  7 2 5 1 6 3 4 2
```

```
Output: 1 2 2 3 4 5 6 7
```

```
Merge / S=1: 17 comparisons
```

```
S=4: 19 • S=8: 22
```

01 Correctness checks passed

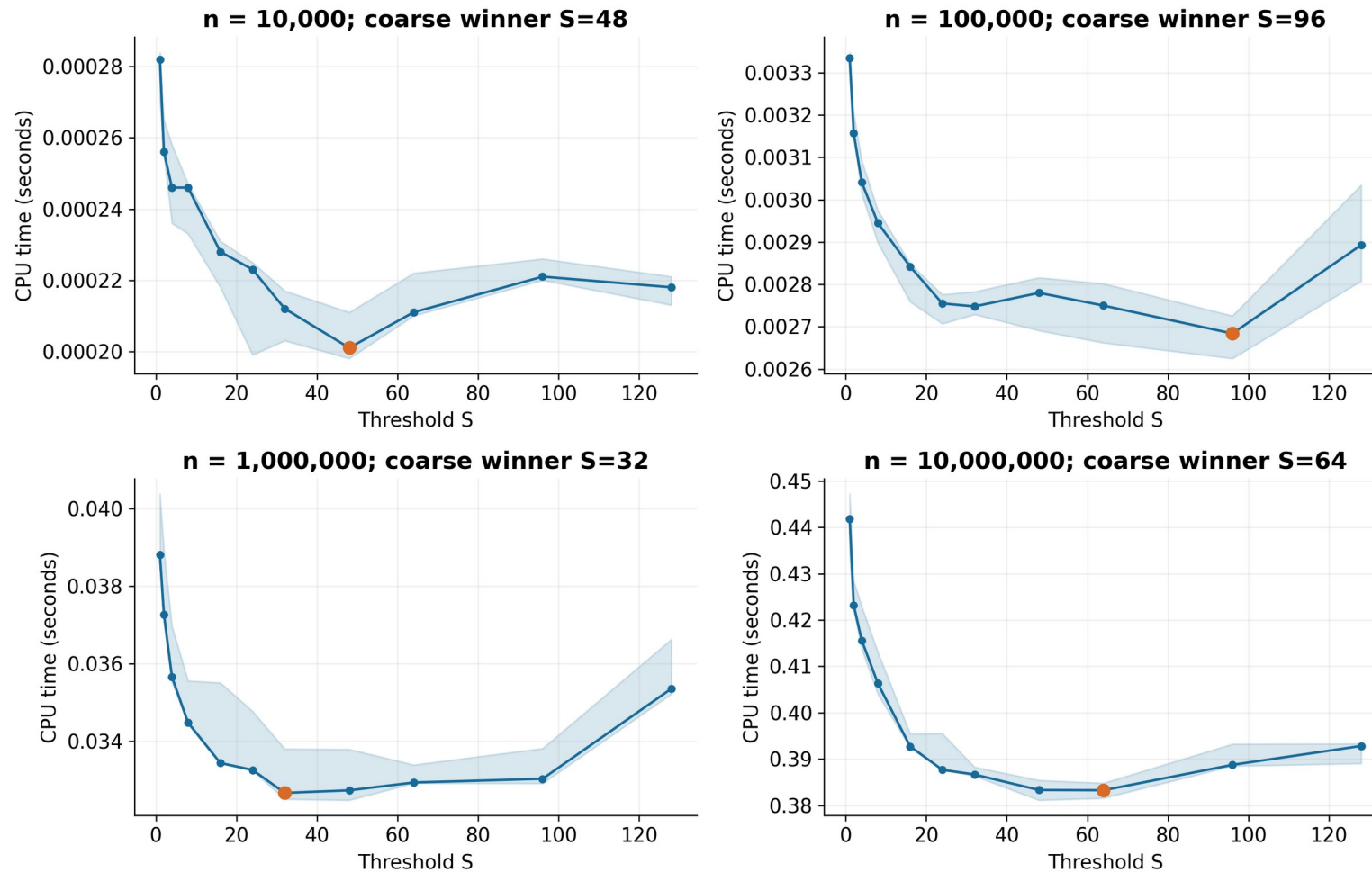
02 Fixed-S growth fits the analysis

03 Faster runtime, more comparisons

Limit: one implementation and machine;
the exact best S is sensitive to timing
variation.

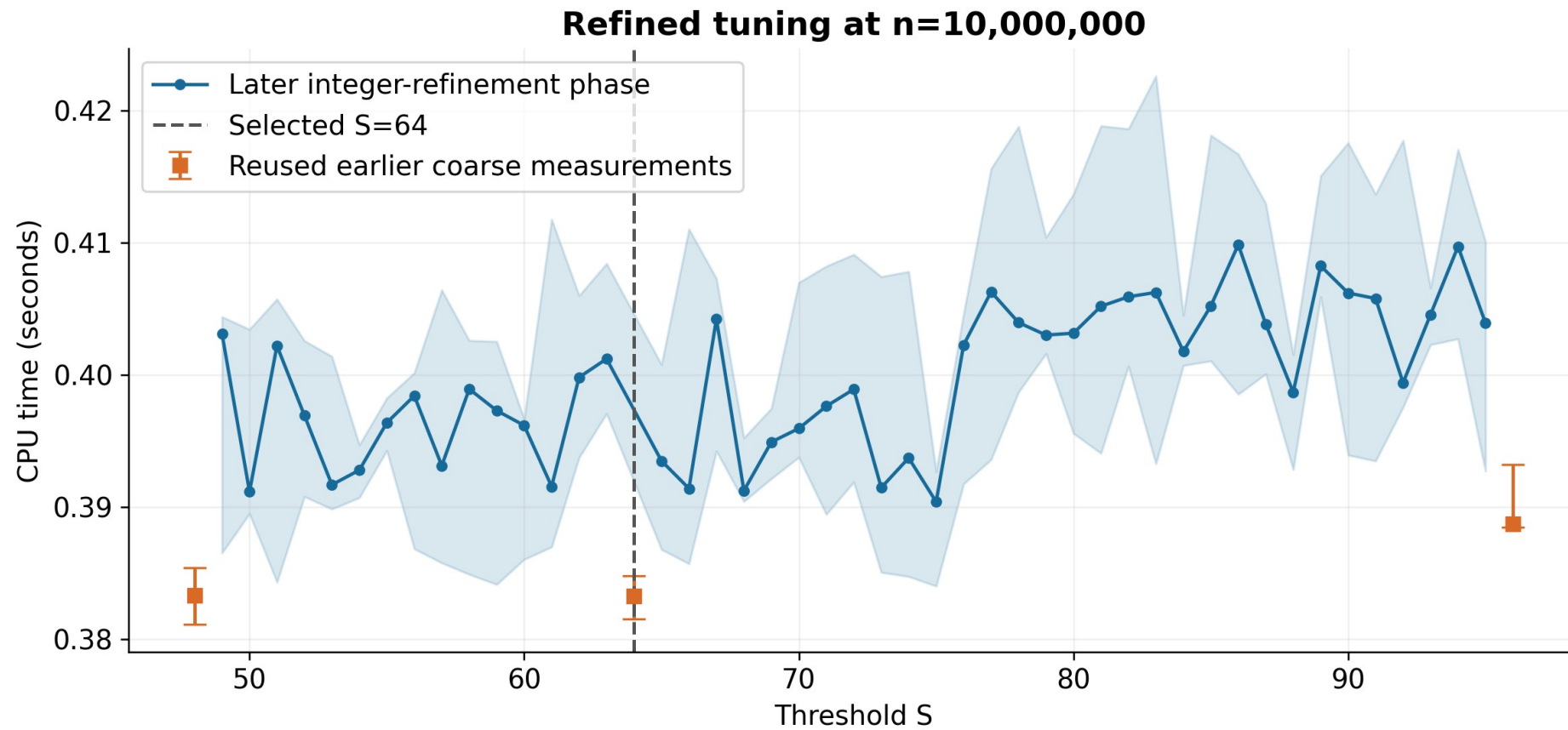
Questions / 2 minutes

CPU time across four input sizes



Median across 5 seeds; shading/error bars = interquartile range

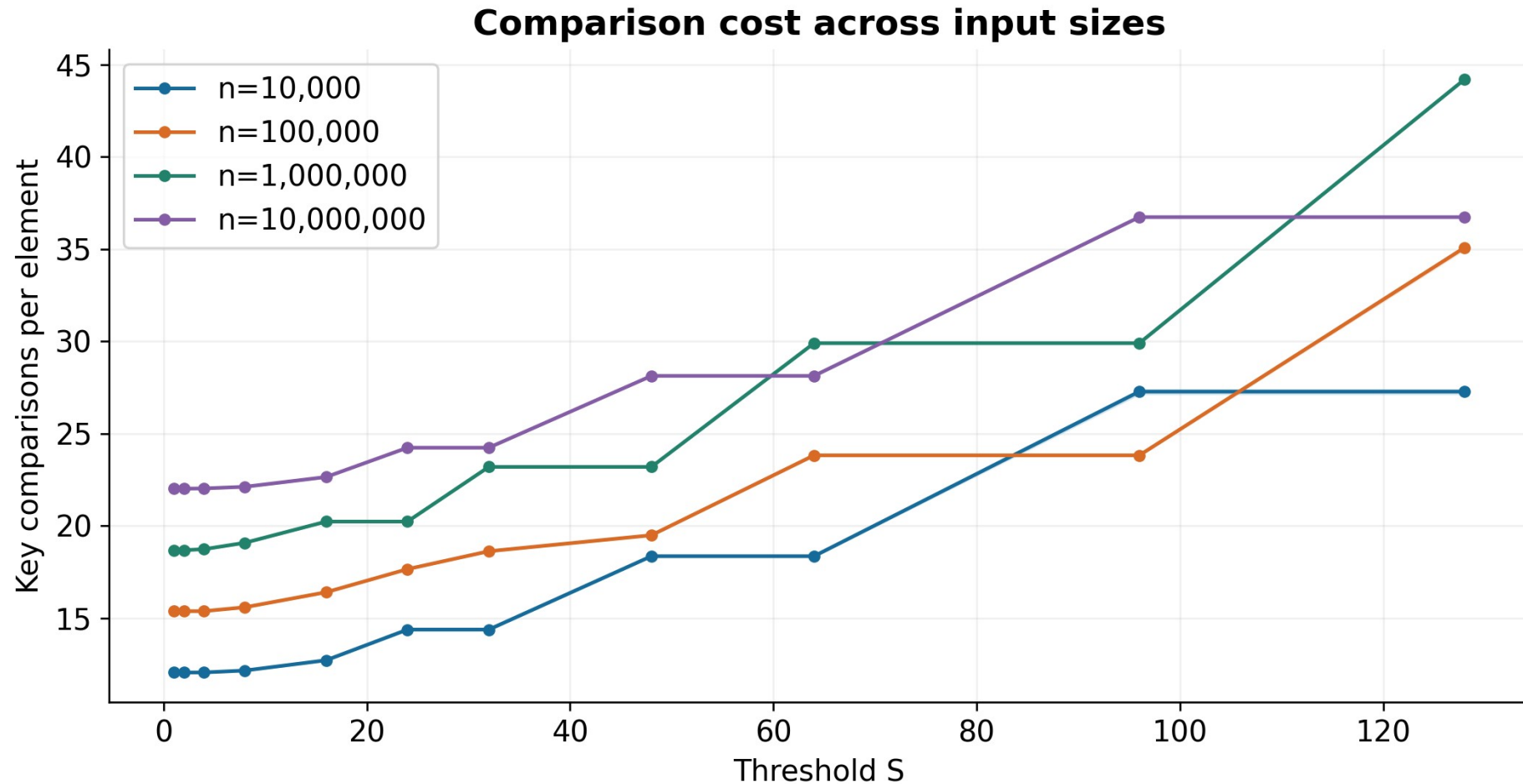
Refinement reveals run-phase variation



Median across 5 seeds; shading/error bars = interquartile range

At n=10m, S=39–75 gives identical leaves (38 or 39 elements). Tiny ranks are not robust.

Key comparisons per element across input sizes



Median across 5 seeds; shading/error bars = interquartile range

Count data comparisons, including the final false one

Insertion example: [2, 1, 3]

$2 > 1 \rightarrow \text{true}; \text{shift } 2$

$2 > 3 \rightarrow \text{false}; \text{stop}$

Total: 2 key comparisons

Merge example: [1, 3] + [2, 4]

$1 \leq 2; 3 \leq 2; 3 \leq 4$

Copy remaining 4 without comparing

Total: 3 key comparisons

Excluded: index checks, loop bounds, copying, and exhaustion checks.

Count=true records uint64 comparisons.

Count=false removes counter increments at compile time.

Correctness and complete experiment coverage

2,020 recorded passes

505 configuration/seed groups; all required sizes and repeats

Algorithm checks

Empty, singleton, odd lengths

Sorted, reversed, equal, duplicate keys

Random arrays and signed extremes

S=1 exactly matches original counts

Independent checks

Exact `std::sort` reference equality

Address + undefined-behavior sanitizers

CSV coverage audit

Threshold recomputed from saved data

Reproducible measurements; explicit limitations

Data & timing

mt19937 + unbiased rejection sampling

Tuning: 101, 202, 303, 404, 505

Validation: 1101, 1202, 1303, 1404, 1505

Three timed repetitions per seed

CPU: std::clock; elapsed: steady_clock

Interpretation limits

Five seeds; IQR is not a confidence interval

Coarse and fine phases occur at different times

Small inputs are timer-resolution sensitive

One machine and uniform input distribution

No alternative-distribution benchmarks claimed

Regenerate: make test → scripts/run_experiments.py → scripts/plot_results.py

Questions we should all be able to answer

Why can insertion sort help?

Quadratic work stays in small leaves; recursion and copying costs fall.

Why do thresholds form plateaus?

Several S values stop at the same recursive subarray sizes.

Why not minimize comparisons?

CPU time also includes movement and control overhead.

Is $S=64$ universally optimal?

No. It is the best observed candidate under this implementation and run.

Does the graph prove the complexity?

No. The derivation establishes the bound; measurements test consistency.